

AgriSense Market Linkage Platform

PRD + TRD + Architecture + Team Execution Plan

PS 26132 | Team of 6 | React Native (Expo) | 2 Days Remaining

PART 1 — PRD (Product Requirements Document)

1.1 Vision

A mobile marketplace that gives smallholder farmers and FPOs price intelligence, quality grading, and direct access to verified buyers — replacing fragmented, information-poor selling with transparent, matched, trackable transactions.

1.2 Problem (from PS, condensed)

Farmers lack visibility into prices, quality standards, buyer demand, and logistics — leading to distress sales and weak bargaining power. Buyers struggle to source consistent, verified volume. No unified platform connects price discovery to an actual transaction.

1.3 Users

Role	Who	Core need
Farmer	Individual smallholder	Know when/where to sell, get a fair matched buyer, get paid reliably
FPO Operator	Staff of a Farmer Producer Org	Combine members' small lots into sellable volume
Buyer	Trader, processor, institutional buyer	Source consistent graded volume from verified sellers
Admin (optional, low priority)	Platform operator	Resolve disputes, verify buyers

1.4 Feature List (mapped 1:1 to the PS's 14 requirements)

Current & expected prices across markets
Localised price trends
Sale-window recommendation
Lot creation
Quality grading (photo-based)
Buyer matching (including multi-lot fulfillment for large orders)
FPO aggregation (combine member lots into one batch)
Buyer demand posting/aggregation
Buyer verification badge
Digital offers (single-lot and multi-lot)
Transport & storage options (informational lookup)
Arrival volumes (data point alongside price)
Payment tracking (Razorpay test-mode escrow simulation)
Dispute/grievance process

1.5 Explicitly Out of Scope

Disease detection, crop recommendation (different PS, don't dilute the pitch)
Real logistics fleet/booking, real cold-chain operations (informational only — see earlier discussion)
Real payout-to-farmer bank transfer (simulated status flip; production would use Razorpay Payouts)
RAG/chat agents (not needed — see note above)
Native geocoding/maps API (use a seeded static list of mandi locations instead — faster, zero API-key risk)

1.6 Success Metrics (mirror the PS's stated outcomes)

Improved price realisation → visible price band + sale-window advice. Reduced info asymmetry → aggregated price/demand/logistics in one place. Lower transaction cost → digital offers replace broker layers. Stronger FPO aggregation → the batch-lot feature. Reduced post-harvest loss → sale-window timing advice. Transparent records → payment/dispute status trail.

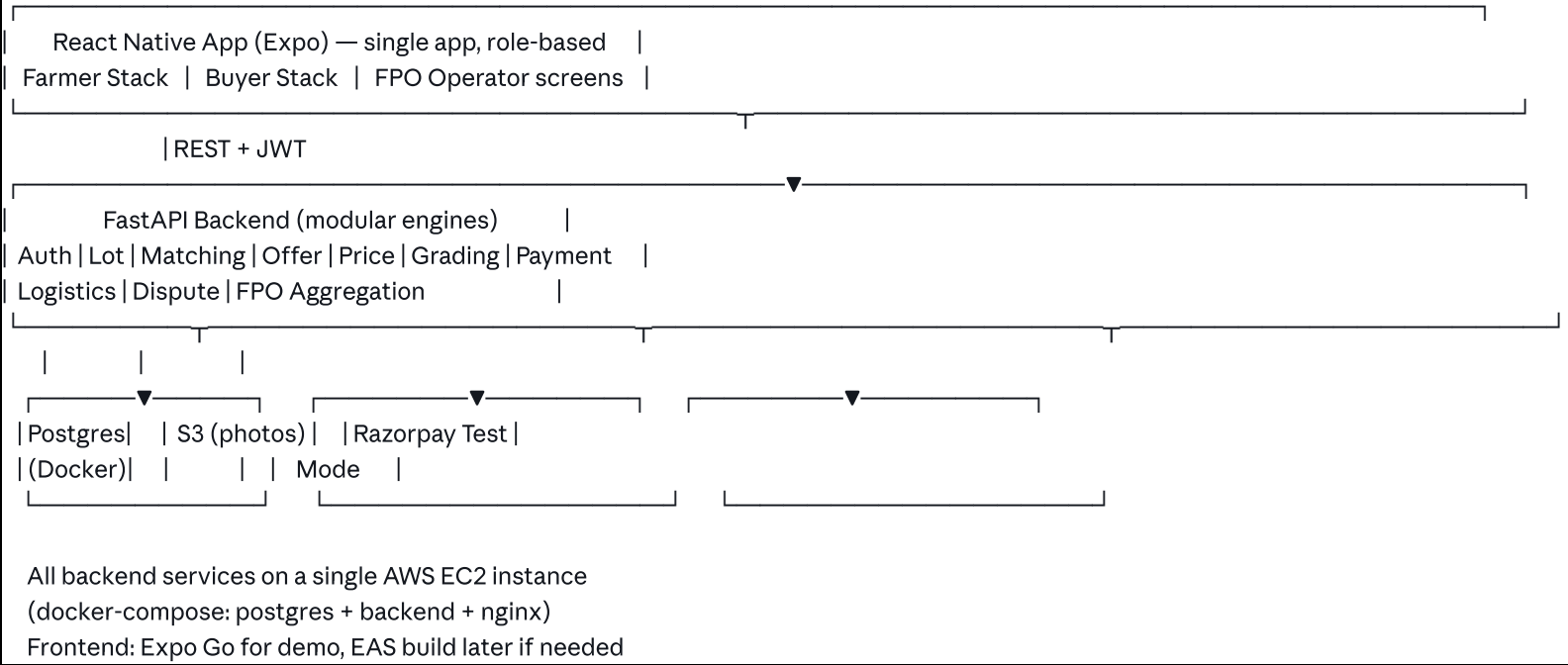
PART 2 — TRD (Technical Requirements Document)

2.1 Tech Stack

Layer	Choice	Why
Mobile app	React Native + Expo	No native build step, fast iteration, Expo Go lets judges scan a QR and run your app on their own phone — strong live-demo advantage
Navigation	React Navigation (bottom tabs + stack)	Standard, well-documented, fast to wire
State/data fetching	TanStack Query (React Query) + React Context for auth	Handles server state/caching without Redux overhead — you don't have time for Redux boilerplate
Backend	FastAPI (Python)	Matches your team's existing skill from AgriSense
DB	PostgreSQL, Dockerized on the same EC2 instance	Skip RDS — saves setup time and cost, not needed for a demo
ORM/migrations	SQLAlchemy + Alembic	
Object storage	AWS S3	Lot photos
Auth	JWT (FastAPI <code>python-jose</code> or <code>fastapi-users</code>)	Simple role-based auth, no need for a full IAM system
Payments	Razorpay (test/sandbox mode)	Explicitly required by the PS's "payment tracking"
AI — pricing	Reused TFT + LightGBM artifacts from your existing AgriSense project	Already trained, already validated — don't rebuild
AI — grading	EfficientNet-B0 (transfer learning, reuse your disease-detection training code, repointed) + OpenCV rule-based fallback	Dual-path for reliability under time pressure
Infra	Single AWS EC2 instance, Docker + docker-compose, Nginx reverse proxy	Fastest path to a stable live demo
No RAG, no agent framework	—	Not needed for this PS's structured-data problem — skip entirely
No live geocoding API	—	Use a seeded static table of Maharashtra mandi locations (name, lat, lng) instead

2.2 System Architecture





2.3 Data Model (final, includes multi-lot support)

❏
users(id, name, phone, role[farmer buyer fpo_admin], lat, lng, verified, created_at)
fpos(id, name, region)
farmers(user_id FK, fpo_id FK nullable)
buyers(user_id FK, company_name, verified_status)
mandi_locations(id, name, district, lat, lng) -- seeded static list, no geocoding API needed
buyer_demand(id, buyer_id FK, crop, desired_qty, desired_grade, max_price, lat, lng)
lots(id, farmer_id FK, fpo_id FK nullable, crop, quantity_qtl, grade, image_url, lat, lng, price_min, price_mid, price_max, status, created_at)
offers(id, buyer_id FK, offered_price_per_qtl, total_quantity, status, created_at)
offer_lots(offer_id FK, lot_id FK, quantity_allocated) -- supports multi-lot fulfillment
transactions(id, offer_id FK, amount, razorpay_order_id, payment_status, created_at)
logistics_providers(id, name, type[storage transport], lat, lng, capacity, contact)
disputes(id, transaction_id FK, raised_by, reason, status, created_at)
batch_lot_members(batch_lot_id FK, source_lot_id FK, quantity_contributed) -- FPO aggregation split
price_forecasts(id, mandi, commodity, date, p10, p50, p90, generated_at) -- cache

2.4 API Contract

Endpoint	Method	Purpose
<code>/auth/register</code> , <code>/auth/login</code>	POST	JWT auth, role-based
<code>/users/me</code>	GET	Current user profile
<code>/mandi-locations</code>	GET	Seeded location list for pickers
<code>/lots</code>	POST	Create lot → auto price band via Price Engine
<code>/lots/{id}</code>	GET	Lot detail
<code>/lots?farmer_id=&status=</code>	GET	Filtered lot list
<code>/lots/{id}/grade</code>	POST (multipart)	Upload photo → returns grade
<code>/lots/{id}/matches</code>	GET	Ranked buyer matches (single or combinable)
<code>/demands</code>	POST/GET	Buyer posts/lists demand

Endpoint	Method	Purpose
<code>/demands/{id}/matches</code>	GET	Multi-lot combination suggestion to fulfill a large demand
<code>/offers</code>	POST	Body: <code>{buyer_id, lots: [{lot_id, quantity_allocated}], offered_price_per_qtl}</code>
<code>/offers/{id}</code>	PATCH	Accept/reject
<code>/offers?farmer_id=&buyer_id=</code>	GET	Offer lists
<code>/transactions/{offer_id}/create-order</code>	POST	Razorpay order creation
<code>/transactions/webhook</code>	POST	Razorpay payment confirmation
<code>/transactions/{id}</code>	GET	Status
<code>/logistics?lat=&lng=&radius=</code>	GET	Nearby storage/transport
<code>/disputes</code>	POST/GET/PATCH	Raise/view/resolve
<code>/fpo/aggregate</code>	POST	Body: <code>{lot_ids: []}</code> → creates batch lot
<code>/fpo/{id}/lots</code>	GET	FPO's member lots
<code>/ai/price-forecast?mandi=&commodity=</code>	GET	Direct quantile access for charts

2.5 AI/ML Pipeline

Price Engine (reuse, don't rebuild): export trained TFT + LightGBM artifacts from your existing AgriSense repo, wrap in a `predict(mandi, commodity, date) → {p10, p50, p90}` function loaded at backend startup. Add a small derived rule on top for the sale-window line ("prices trending up X% over 7 days — consider holding").

Grading Engine (dual-path):

Path A (build first, guaranteed): OpenCV rule-based — blemish/discoloration detection via color thresholding + contour analysis → grade from % surface area affected.

Path B (stretch): fine-tune EfficientNet-B0 on a sourced dataset (see resources below), reusing your existing disease-detection training pipeline with a new 3-class head (Grade A/B/C).

Backend tries the model first, falls back to rule-based if unavailable/low-confidence.

2.6 External Resources & Data Sources

Resource	Use	Link
Existing AgriSense trained model artifacts	Price Engine — reuse directly, don't retrain your own repo	
Agmarknet historical data (already scraped previously)	Price training data — reuse, don't re-scrape under time pressure	https://agmarknet.gov.in/
NASA POWER API	Optional weather enrichment if time allows — not critical this round	https://power.larc.nasa.gov/
Kaggle — Fruit & Vegetable Quality (Fresh/Rotten), 14 types incl. vegetables	Primary grading dataset — broadest crop coverage	https://www.kaggle.com/datasets/zlatan599/fruitquality1
Kaggle — Fresh and Stale Fruits & Vegetables (incl. tomato, capsicum)	Alt/supplement — more India-relevant crops	https://www.kaggle.com/datasets/raghavrpotdar/fresh-and-stale-images-of-fruits-and-vegetables

Resource	Use	Link
Kaggle — Fruits Fresh and Rotten (apple/banana/orange)	Well-tested baseline if you need a fast first working model	https://www.kaggle.com/datasets/sriramr/fruits-fresh-and-rotten-for-classification
Razorpay test mode docs	Payment integration reference	https://razorpay.com/docs/payments/payments/test-mode/
Expo docs	RN app setup	https://docs.expo.dev/
React Navigation	Navigation setup	https://reactnavigation.org/

Buyer demand, logistics providers, mandi locations: you will not find real datasets for these — **seed them yourselves** with realistic fake data (e.g. 15–20 buyers with varied crop/price/location, 10–15 logistics providers across a few Maharashtra districts, 20–30 real mandi names with real lat/long). This is normal and expected for a hackathon demo — don't spend time hunting for a "real" dataset that doesn't exist.

PART 3 — Frontend Architecture (React Native / Expo)

3.1 Why Expo specifically

Zero native build step during development, hot reload, and — most importantly — **Expo Go lets you demo live on a judge's own phone by having them scan a QR code**, which is a genuinely strong, memorable demo moment most teams won't have.

3.2 Navigation Structure

RootNavigator AuthStack (unauthenticated) RoleSelectScreen LoginScreen RegisterScreen FarmerTabs (role = farmer) HomeTab MyLotsTab OffersTab FPOTab (visible only if fpo_id set, or role = fpo_admin) ProfileTab BuyerTabs (role = buyer) BrowseTab DemandTab OffersTab PaymentsTab ProfileTab
--

3.3 Farmer Side — Page-by-Page

Screen	Key components
Home	PriceTickerCard (today's price for their main crop), SaleWindowBanner (hold/sell advice), QuickStats (active lots, pending offers count)
My Lots (list)	LotCard list with status chips (Created / Matched / Offered / Sold)
Create Lot	Crop dropdown, quantity input, MandiLocationPicker (from seeded list — no maps API), photo upload button → triggers grading
Lot Detail	PriceBandChart, GradeBadge (result + confidence), MatchedBuyersList, ReceivedOffersList, LogisticsList (nearby storage/transport), PaymentTrackerStepper (once sold)

Screen	Key components
Offers Inbox	List of received offers per lot, Accept/Reject buttons
Offer Detail	Full offer info; if multi-farmer batch, shows this farmer's allocated share
FPO Dashboard (FPO operator only)	List of member farmers' ungrouped lots with checkboxes
Create Batch Lot	Selected lots summary, summed quantity, "Create Batch Lot" button
Profile	Basic info, logout
Disputes	List + "Raise Dispute" form

3.4 Buyer Side — Page-by-Page

Screen	Key components
Browse Lots	Filterable LotCard list (crop, grade, location radius, price range)
Lot Detail	Grade, price, farmer/FPO name, distance, LogisticsList, "Send Offer" button
My Demands (list)	List of posted demand requests with status
Post Demand	Crop, desired quantity, desired grade, max price, location
Demand Matches	Shows the multi-lot combination that fulfills a large order ("your 10 quintals = FPO batch 5qtl + Farmer A 3qtl + Farmer B 2qtl"), single "Send Offer for Full Order" button
Sent Offers	List with status
Offer Detail	Single or multi-lot breakdown
Payment Tracker	Razorpay checkout trigger, escrow status stepper (Pending → Held → Released)
Profile	Verification badge status
Disputes	List + raise new

3.5 Shared Components (build once, use everywhere — assign ownership Day 1 hour 1 so it isn't duplicated)

`LotCard`, `PriceBandChart`, `GradeBadge`, `MatchScoreBar`, `OfferStatusStepper`, `LogisticsList`, `DisputeForm`, `MandiLocationPicker`

PART 4 — Team Assignment (Zero Overlap)

Person	Role	Owns
B1	Backend	Auth, Lot Engine, Matching Engine (incl. multi-lot combination logic), Buyer-demand endpoints, FPO Aggregation Engine, DB schema (final authority)
B2	Backend	Offer Engine (incl. <code>offer_lots</code>), Payment Engine (Razorpay), Logistics Engine, Dispute Engine, AWS/EC2 deployment ownership
A1	AI	Price Engine integration, sale-window logic, arrival-volume data feed

Person	Role	Owns
A2	AI	Grading Engine (rule-based + CV model), all seed data (buyers, logistics providers, mandi locations)
F1	Frontend	Entire Farmer stack (Home, My Lots, Create Lot, Lot Detail, Offers Inbox, FPO screens)
F2	Frontend	Entire Buyer stack (Browse, Demand, Offers, Payments) + owns the shared component library (built Day 1 hour 1, both F1/F2 agree on props before building screens)

Every one of the 14 PS features has exactly one backend owner and one frontend owner listed above — no two people should ever touch the same endpoint or the same screen file.

PART 5 — 2-Day Hour-by-Hour Schedule

Day 1

Hour 0-1 (whole team): Lock the DB schema (section 2.3) and API contract (section 2.4) together — nobody codes before this is agreed, since it's what lets all 6 of you work in parallel without collisions. Confirm shared component props (F1+F2).

Hour 1-2 (whole team): Repo setup (monorepo: `/backend`, `/app`), EC2 provisioning (B2), Expo project init (F1/F2 together), export price model artifacts into new repo (A1).

Hour 2-8:

B1: Auth + Lot + Matching endpoints

B2: Offer + Payment (Razorpay test order creation) + Docker/EC2 skeleton deployed early

A1: `/ai/price-forecast` live and returning real quantiles

A2: Rule-based grading working end-to-end (non-negotiable by end of day); start CV model fine-tuning in parallel; generate all seed data

F1: Auth screens + Create Lot + My Lots list

F2: Shared component library + Browse Lots + Lot Detail (buyer view)

End of day (30 min, whole team): Demo what's stitched together, fix integration mismatches immediately.

Day 2

Hour 0-5:

B1: FPO Aggregation endpoint, multi-lot matching for `/demands/{id}/matches`

B2: Razorpay webhook + escrow status flow, Dispute + Logistics endpoints

A1: Sale-window recommendation logic, price trend chart data

A2: Finalize grading model (or confirm rule-based is production-ready for demo), wire into `/lots/{id}/grade`

F1: Lot Detail full wiring (grading result, matches, offers), FPO screens, Disputes

F2: Demand posting, multi-lot demand-matches screen, Offer flow, Payment Tracker with real Razorpay checkout

Hour 5-7 (whole team): Full integration — every screen wired to real endpoints, run the entire

farmer→grade→match→offer→payment→dispute journey end-to-end 2-3 times. **Freeze new features. Bug fixes only from here.**

Hour 7-8: Deploy final build, confirm Expo Go QR demo works on a second phone, build the pitch deck, rehearse with a timer, record a backup demo video.

Bottom line: every feature in the PS has exactly one owner on each side, the schema and API contract are locked before anyone writes feature code, and the riskiest pieces (grading model, Razorpay webhook) have simple fallbacks built in from hour one — not scrambled together on the last night.